Integración OAuth 2.0 / Microsoft Entra ID — Miharu+ Desktop (DDO)

Fecha: Febrero 2026 **Versión:** 1.1 **Referencia:** Exigencias Mínimas Módulos de Seguridad v3.20251001

---

1. Contexto y Exigencia

La Dirección de Ciberseguridad Corporativa exige que todo sistema de información se integre con la plataforma de **Gobierno de Identidades (Microsoft Entra ID)** bajo los siguientes términos:

Requisito	Estándar exigido	Prioridad
Autenticación	OAuth 2.0 / OpenID Connect (OIDC) / SAML 2.0	**Obligatorio**
Aprovisionamiento	SCIM 2.0 o JIT (Just In Time)	Obligatorio / Alternativa válida
Logs de auditoría	6 meses online — 3 años backup	Obligatorio
Timeout por inactividad	Parametrizable	Obligatorio
Reportes exportables	Por perfil y por usuario	Obligatorio
Módulo de admin de usuarios	Modular, flexible, por menú (no CLI)	Obligatorio

> **Nota clave del documento:** "*Toda aplicación que requiera entregar la administración de usuarios a nuestra área deberá integrarse con nuestro modelo estándar de autenticación y aprovisionamiento, en caso de no cumplir con este requisito no se podrá recibir la administración de la aplicación.*"

---

2. Estado Actual del Sistema

2.1 Stack tecnológico

- **.NET Framework 4.8** — C# y VB.NET (codebase mixto)
- **WinForms** — cliente desktop (aplicación principal DDO)
- **ASP.NET ASMX** — Web Services para autenticación y lógica de negocio
- **SQL Server** — repositorio de usuarios, perfiles y permisos

2.2 Arquitectura de autenticación actual

```
[FormLogin.vb]  →  [Program.vb]  →  [SecurityWebService / SecurityDMZWebService]
                                              ↓
                                    CrearCanalSeguro()     ← RSA key exchange
                                    ValidateUser()         ← user/pass cifrados
                                    FillSession()          ← carga permisos
```

```
                                        ↓
                                Objeto Sesion / Usuario (en memoria)
                                        ↓
                                [FormMain] — aplicación operativa
```

Proyectos clave de seguridad:

Proyecto	Rol
`Miharu.Security.WebService`	Servicio ASMX — autenticación, sesiones, LDAP
`Miharu.Security.WebService.DMZ`	Variante para acceso externo/DMZ
`Miharu.Security.Library`	Wrapper cliente + modelo de sesión (`Sesion`, `Usuario`, `PerfilManager`)
`Miharu.Security.DataAccess`	Acceso a BD — stored procedures de seguridad
`Miharu.Desktop`	Aplicación WinForms — `FormLogin.vb`, `Program.vb`

2.3 Características positivas del sistema actual

- Sistema de perfiles y permisos granular ya implementado (6 tipos de permiso por recurso)
- Objeto `Sesion`/`Usuario` bien estructurado — apto para recibir claims externos
- Soporte de modo Interno / Externo (DMZ) ya contemplado — facilita el modo híbrido
- LDAP configurado en el WebService (`latam.brinksgbl.com`) — actualmente deshabilitado

2.4 Dato relevante — LDAP corporativo

El Web Service tiene configurado el directorio corporativo:

```
<add key="LDAP.Validar"    value="false" />
<add key="LDAP.ServerPath"
value="LDAP://latam.brinksgbl.com/DC=latam,DC=brinksgbl,DC=com" />
```

> Si `latam.brinksgbl.com` está sincronizado con Entra ID vía **Azure AD Connect**, los usuarios corporativos ya existen en Entra ID — lo que simplifica el aprovisionamiento JIT porque los claims del token ya tendrán datos consistentes.

2.5 Brechas frente al documento de exigencias

Requisito	Estado actual	Brecha
OAuth 2.0 / OIDC	❌ Auth propia RSA + ASMX	Alta
Microsoft Entra ID	❌ Sin integración	Alta
Aprovisionamiento automático	❌ Solo manual vía módulo	Alta
SCIM 2.0	❌ No implementado	Alta

Requisito	Estado actual	Brecha
Logs auditoría 6m / 3a	⚠ Parcial (`TBL_Usuario_Sesion`)	Media
Timeout inactividad	⚠ Parcial (Forms Auth timeout)	Media
Reportes exportables	☑ Ya implementado	Cumple
Módulo admin usuarios	☑ Ya implementado	Cumple

---

3. Propuesta de Integración

3.1 Estrategia: Modo híbrido con JIT Provisioning

Mantener el flujo de autenticación actual y **agregar Entra ID como ruta paralela**, usando **JIT (Just In Time) Provisioning**, opción válida y explícitamente aceptada por el documento de exigencias.

Esto permite:

- No romper la operación actual durante la transición
- Soportar usuarios que aún no están en Entra ID
- Hacer una migración progresiva

3.2 Flujo propuesto

```
Usuario abre la aplicación (WinForms)
            │
            ▼
     ┌──────────────────────┐
     │     FormLogin.vb     │
     │  ┌────────────────┐  │
     │  │ Usuario/Clave  │  │  ← Flujo ACTUAL (se conserva)
     │  └────────────────┘  │
     │                      │
     │  ┌────────────────┐  │
     │  │ Acceso         │  │  ← NUEVO botón
     │  │ Corporativo    │  │
     │  └────────────────┘  │
     └──────────────────────┘
            │
     ┌──────┴──────────────────────────┐
     │                                 │
     ▼                                 ▼
[Flujo actual]                  [NUEVO: Entra ID / OAuth 2.0]
SecurityWebService               MSAL.NET interactivo
.ValidateUser()                  → Browser Microsoft Login (MFA)
     │                           → Token JWT de Entra ID
     │                           → ValidateEntraIDUser() en WebService
     │                           → JIT: buscar/crear usuario en SQL
     │                           → Cargar permisos desde TBL_Perfil
     │                                       │
     └──────────────┬──────────────────────┘
                    ▼
```

```
            Objeto Sesion / Usuario
            (mismo contrato — sin cambios en FormMain)
                     ▼
            Aplicación operativa (sin cambios)
```

3.3 Mapeo de Claims de Entra ID al modelo actual

Claim del Token JWT	Campo en `Usuario` / `TBL_Usuario`
`oid`	Campo nuevo: `EntraIDObjectId` (GUID único)
`preferred_username` / `upn`	`Login`
`given_name`	`Nombres`
`family_name`	`Apellidos`
`email`	Email del usuario
`name`	`NombreCompleto`
`roles` (App Roles)	Mapeo directo a `TBL_Perfil` — viene en el token, sin llamadas extra

Ejemplo del token JWT que recibiremos con App Roles configurados:

```json
{
  "oid": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "preferred_username": "juan.perez@banco.com",
  "given_name": "Juan",
  "family_name": "Pérez",
  "name": "Juan Pérez",
  "roles": ["Operador"]
}
```

---

4. Componentes a Implementar

4.1 Nuevos componentes

Componente	Proyecto	Descripción
`EntraIDAuthService.cs`	`Miharu.Security.Library`	Encapsula MSAL.NET: autenticación interactiva, validación de token, mapeo de claims
`EntraIDConfig.cs`	`Miharu.Security.Library`	Modelo de configuración (TenantId, ClientId, Scopes, RedirectUri)
`ValidateEntraIDUser()`	`Miharu.Security.WebService`	Nuevo método ASMX: recibe token, valida, ejecuta JIT, retorna sesión

Componente	Proyecto	Descripción
`JITProvisioningService.cs`	`Miharu.Security.DataAccess`	Lógica DB: buscar/crear usuario, consultar tabla de homologación de roles
`TBL_EntraID_Rol_Perfil` (BD)	`Miharu.Security.DataAccess`	Tabla de homologación: mapea roles de Entra ID a perfiles de Miharu

4.2 Componentes a modificar

Componente	Cambio
`FormLogin.vb`	Agregar botón "Acceso Corporativo" + llamada async
`Program.vb`	Agregar `IniciarSesionEntraID()` como tercer flujo de entrada
`App.config`	Agregar sección de configuración Entra ID (sin mapeo de roles — ese va en BD)
`TBL_Usuario` (BD)	Agregar columna `EntraIDObjectId` (nullable)

4.3 Tabla de homologación de roles

El mapeo entre los roles de Entra ID y los perfiles de Miharu **no va en el `App.config`** sino en una tabla de BD, ya que los nombres de roles en Entra ID no necesariamente coinciden con los códigos de perfiles en `TBL_Perfil` y pueden cambiar sin requerir un redeploy.

```
CREATE TABLE TBL_EntraID_Rol_Perfil (
    Id              INT IDENTITY PRIMARY KEY,
    EntraIDRole     VARCHAR(100) NOT NULL,  -- valor exacto del claim "roles" del
token JWT
    PerfilId        INT NOT NULL,           -- FK a TBL_Perfil
    Descripcion     VARCHAR(200) NULL,
    Activo          BIT NOT NULL DEFAULT 1,
    FechaCreacion   DATETIME NOT NULL DEFAULT GETDATE(),
    UsuarioCreacion VARCHAR(100) NOT NULL,
    CONSTRAINT FK_EntraIDRol_Perfil FOREIGN KEY (PerfilId) REFERENCES
TBL_Perfil(Id)
)
```

Ejemplo de datos iniciales (los valores exactos de `EntraIDRole` los confirma el banco):

EntraIDRole	Perfil Miharu	Descripción
`Admin`	ADMIN_DDO	Administrador del sistema
`Operador`	OPERADOR_DDO	Operación estándar
`Consulta`	CONSULTA_DDO	Solo lectura

> **Ventaja:** si el banco cambia el nombre de un rol o agrega uno nuevo, solo se hace un `INSERT`/`UPDATE` en esta tabla — sin recompilar ni redesplegar la aplicación.

4.4 Configuración en App.config

```
<!-- Microsoft Entra ID / OAuth 2.0 -->
<add key="EntraID.Enabled"       value="true" />
<add key="EntraID.TenantId"      value="[TENANT-ID del banco — proporcionado por
Azure team]" />
<add key="EntraID.ClientId"      value="[CLIENT-ID — generado al registrar la
app]" />
<add key="EntraID.Authority"     value="https://login.microsoftonline.com/[TENANT-
ID]" />
<add key="EntraID.Scopes"        value="openid profile email" />
<add key="EntraID.RedirectUri"   value="http://localhost" />
<!-- Perfil base si el usuario llega sin rol asignado en Entra ID -->
<add key="EntraID.DefaultPerfil" value="[Código del perfil base en TBL_Perfil]" />
```

> **Nota:** el mapeo de roles ya no está en el `App.config` — vive en `TBL_EntraID_Rol_Perfil`. Con App Roles los scopes tampoco incluyen `User.Read` — todo lo necesario viene en el token JWT.

4.4 Paquetes NuGet requeridos

Paquete	Compatibilidad	Uso
`Microsoft.Identity.Client`	.NET 4.5+	MSAL.NET — flujo OAuth 2.0 / OIDC interactivo
`Microsoft.IdentityModel.Tokens`	.NET 4.5+	Validación de token JWT
`System.IdentityModel.Tokens.Jwt`	.NET 4.5+	Parseo y lectura de claims del JWT

Todos son compatibles con **.NET Framework 4.8**.

> **Con App Roles NO es necesario** instalar el SDK de Microsoft Graph ni agregar el paquete `Microsoft.Graph`. Los roles llegan directamente en el token JWT.

---

5. Lógica JIT Provisioning

El aprovisionamiento JIT es la alternativa válida definida en el documento cuando no se implementa SCIM 2.0. Con App Roles, el token ya trae el rol del usuario sin necesidad de consultas adicionales a Microsoft Graph. El mapeo a perfiles de Miharu se resuelve mediante la tabla de homologación `TBL_EntraID_Rol_Perfil`.

```
Token de Entra ID recibido en el WebService
            │
            ▼
Extraer claims del JWT:
  oid, upn, given_name, family_name, email, roles[]
            │
```

```
            ▼
Homologación de rol:
  SELECT PerfilId FROM TBL_EntraID_Rol_Perfil
  WHERE EntraIDRole = roles[0] AND Activo = 1
            │
      ┌─────┴──────────┐
  Encontrado        No encontrado
    │                    │
    ▼                    ▼
  Usar PerfilId      Usar perfil base (EntraID.DefaultPerfil)
  de la tabla        + registrar advertencia en log:
                     "Rol sin homologación: [rol]"
            │
            ▼
¿Existe usuario en TBL_Usuario WHERE EntraIDObjectId = oid?
    │                           │
   SÍ                          NO
    │                           │
    ▼                           ▼
Actualizar datos             INSERT nuevo usuario
si cambiaron                 con datos del token
(nombre, email, perfil)      Asignar perfil homologado
    │                           │
    └──────────┬────────────────┘
               ▼
Cargar permisos desde TBL_Perfil
               ▼
Construir objeto Sesion → retornar VALIDO
               ▼
Registrar en log: "Alta/acceso JIT via Entra ID — Rol: [rol] → Perfil: [perfil]"
```

> **Ventaja clave:** no se hace ninguna llamada a Microsoft Graph API durante el login, y el mapeo de roles es flexible — se gestiona desde BD sin recompilar la aplicación.

---

6. Registro de la App en Microsoft Entra ID

Responsable: Equipo Azure/Infraestructura del banco

6.1 Pasos de registro

Paso	Acción	Detalle
1	Nueva registro	Azure Portal → **Entra ID** → **App Registrations** → **New Registration**
2	Nombre	`Miharu Desktop DDO` (u otro nombre que defina el banco)
3	Tipo de cuenta	**Single tenant** — Accounts in this organizational directory only
4	Redirect URI	Plataforma: **Public client/native** → URI: `http://localhost`
5	Authentication	Habilitar **Allow public client flows = Yes**

Paso	Acción	Detalle
6	Token configuration	Agregar claims opcionales: `email`, `family_name`, `given_name`
7	API permissions	`Microsoft Graph → openid`, `profile`, `email` (delegated)
8	Admin consent	Hacer **Grant admin consent** para el tenant

> **Por qué Public client / sin secret:** la aplicación es ClickOnce/desktop. Una app web puede guardar un "client secret" de forma segura en el servidor, pero una app de escritorio no — por eso se usa el flujo público donde el usuario se autentica directamente con Microsoft. **El `ClientId` es un GUID generado por Azure, no tiene relación con el nombre del ensamblado ni del ejecutable.**

6.2 Configuración de App Roles

En lugar de usar grupos de Active Directory (que requeriría llamadas a Microsoft Graph), el banco debe configurar **App Roles** directamente en el registro de la aplicación. El rol llega dentro del token JWT, simplificando el código y eliminando permisos adicionales.

Ruta: App Registrations → `Miharu Desktop DDO` → **App roles** → Create app role

Roles a crear:

Display name	Value (exacto)	Descripción	Assigned to
Administrador	`Admin`	Acceso total al sistema	Users/Groups
Operador	`Operador`	Operación estándar	Users/Groups
Consulta	`Consulta`	Solo lectura	Users/Groups

> **Importante:** el campo **Value** es el que llega en el claim `roles` del token. Debe coincidir exactamente con lo configurado en `App.config` (`EntraID.Role.Admin`, etc.). Coordinar estos nombres antes de implementar.

6.3 Datos que debe entregar el equipo Azure al proyecto

Dato	Dónde encontrarlo
`TenantId`	Azure Portal → Entra ID → Overview
`ClientId`	Azure Portal → App Registrations → `Miharu Desktop DDO` → Overview
Nombres exactos de los App Roles creados	App Registrations → App roles → columna Value

6.4 Consideración sobre LDAP / AD Connect

Confirmar con el equipo del banco si el dominio `latam.brinksgbl.com` (AD corporativo ya configurado en el WebService) está sincronizado con Entra ID vía **Azure AD Connect**. Si es así, los usuarios ya existen en Entra ID y el JIT solo necesita hacer un lookup por UPN.

---

7. Tareas de Implementación

- Implementar `EntraIDAuthService.cs` + integración MSAL.NET
- Implementar `ValidateEntraIDUser()` en el WebService
- Implementar lógica JIT + tabla `TBL_EntraID_Rol_Perfil` + columna `EntraIDObjectId` en BD
- Modificar `FormLogin.vb` + `Program.vb`
- Actualizar `App.config` con parámetros de Entra ID
- Pruebas integradas en ambiente de laboratorio (requiere datos del banco)

---

8. Puntos Críticos y Riesgos

8.1 Dependencias externas (bloqueantes)

Bloqueante	Acción requerida
Registro de la app en Entra ID	Solicitar al equipo Azure del banco **con urgencia** — sin `ClientId` y `TenantId` no se puede probar
Nombres exactos de los App Roles	El banco debe definir y confirmar el campo **Value** de cada App Role antes de programar el mapeo
Usuario de prueba con cuenta corporativa	Necesario para validar el flujo completo en ambiente de laboratorio
Confirmación de AD Connect (LDAP sync)	Determina si los usuarios ya existen en Entra ID
Definición de perfil base para JIT	Decisión de negocio: ¿qué perfil asignar si el usuario no tiene App Role asignado?

8.2 Puntos técnicos a resolver

Punto	Detalle
Async en WinForms	`FormLogin.vb` es VB.NET — el flujo MSAL es async/await. Manejar correctamente el hilo de UI para no bloquear la ventana durante el login
Tabla de homologación de roles	Los valores exactos del campo `Value` de los App Roles (confirmados por el banco) deben cargarse en `TBL_EntraID_Rol_Perfil` antes de las pruebas. Si un rol llega sin registro en la tabla, el usuario recibe el perfil base definido en `EntraID.DefaultPerfil` y se registra una advertencia en el log — sin bloquear el acceso
Prevención de login duplicado	El sistema actual valida IP duplicadas. Definir si aplica igual para usuarios de Entra ID
Mapeo de unidades de red	Post-login se mapean drives de red. Confirmar si esto funciona igual con usuarios autenticados por Entra ID

Punto	Detalle
Ambiente de pruebas aislado	El documento exige pruebas en laboratorio con datos diferentes a producción

8.3 Requisitos adicionales del documento que necesitan revisión

Independientemente de la integración con Entra ID, el documento exige estos puntos que deben validarse:

- **Logs de auditoría — retención 6 meses (online) y 3 años (backup)** Exigido en: *Normas generales, punto 1, segundo bullet* — "Estos logs deben ser de dos tipos: por consulta en línea y de respaldo, el periodo de conservación debe ser de al menos 6 meses y 3 años respectivamente." Acción: verificar si `TBL_Usuario_Sesion` y los logs actuales cumplen con la retención requerida.
- **Timeout de inactividad parametrizable** Exigido en: *Normas generales, punto 1, tercer bullet* — "Los sistemas de información del banco deben permitir parametrizar la desconexión de los usuarios por cierto tiempo de inactividad." Acción: revisar si el cliente WinForms implementa esta funcionalidad y si es configurable.
- **Manual técnico del módulo de seguridad** Exigido en: *Normas generales, punto 2* — "Todo desarrollo de software debe poseer documentación formal dentro de la cual, debe existir el manual del módulo de seguridad en medio magnético (Documento técnico)..." con los ítems 2.1 a 2.5.12. Acción: elaborar el documento técnico con índice, políticas, descripción de campos, flujos de creación/modificación/eliminación de usuarios y perfiles.
- **Matriz de control de accesos por perfil** Exigido en: *Normas generales, punto 7* — "Todo aplicativo antes de pasar a producción debe incluir una matriz de control de accesos en la cual se detallen las opciones por perfil, su funcionalidad y los roles con su perfil asociado, las cuales deben estar validadas y aprobadas por el departamento de Seguridad de la información y Ciberseguridad." Acción: elaborar la matriz y enviarla a Seguridad de la Información para aprobación antes del pase a producción.